Indian Institute of Technology Jodhpur

Department of Computer Science and Engineering

CSL 7640: Natural Language Understanding

Assignment 2 Report

Problem 1

Word Embeddings from IIT Jodhpur Data

Problem 2

Character-Level Name Generation using RNN Variants

Submitted to

Prof. Anand Mishra

Department of Computer Science and Engineering
Indian Institute of Technology Jodhpur

Course: CSL 7640 — Natural Language Understanding

Deadline: March 25, 2026

0 Contents

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data **2**
1.1 Task 1: Dataset Preparation . . . 2
1.1.1 Data Sources . . . 2
1.1.2 Preprocessing Pipeline . . . 2
1.1.3 Dataset Statistics . . . 3
1.2 Task 2: Model Training . . . 3
1.2.1 Model Architectures . . . 3
1.2.2 Hyperparameter Experiments . . . 4
1.2.3 Analysis of Hyperparameter Effects . . . 5
1.3 Task 3: Semantic Analysis . . . 5
1.3.1 Cosine Similarity — Top-5 Nearest Neighbours . . . 5
1.3.2 Analogy Experiments . . . 6
1.4 Task 4: Visualisation . . . 7

2 Problem 2: Character-Level Name Generation Using RNN Variants **9**
2.1 Task 0: Dataset . . . 9
2.1.1 Dataset Generation . . . 9
2.1.2 Data Cleaning . . . 9
2.2 Task 1: Model Implementation . . . 10
2.2.1 Common Setup . . . 10
2.2.2 Model 1: Vanilla RNN . . . 10
2.2.3 Model 2: Bidirectional LSTM (BLSTM) . . . 11
2.2.4 Model 3: RNN with Basic Attention . . . 11
2.2.5 Architecture Summary . . . 13
2.2.6 Training Results . . . 13
2.3 Task 2: Quantitative Evaluation . . . 13
2.3.1 Metrics . . . 13
2.3.2 Generation Settings . . . 14
2.3.3 Quantitative Results . . . 14
2.4 Task 3: Qualitative Analysis . . . 15
2.4.1 Representative Generated Samples . . . 15
2.4.2 Realism Analysis . . . 15
2.4.3 Failure Mode Taxonomy . . . 16
2.4.4 Why Vanilla RNN Outperforms . . . 16

3 Conclusions **17**
3.1 Problem 1 Summary . . . 17
3.2 Problem 2 Summary . . . 17

1 Problem 1: Learning Word Embeddings from IIT Jodhpur Data

Objective: Train the Word2Vec Word2Vec models (CBOW and Negative sample Skip-gram with textual information that was retrieved at IIT Jodhpur web. read and process the semantics of the learned embeddings.

1.1 Task 1: Dataset Preparation

1.1.1 Data Sources

An analysis of 19 IIT Jodhpur web pages was done to acquire textual data. spanning: BeautifulSoup Python libraries requests and BeautifulSoup Python libraries.

- **Faculty Profile Pages (6 profiles):**Publications, areas of interest, research. and work experience at the People directory.
- **Department Pages:** Electrical Computer Science & Engineering. Mechanical Engineering, Engineering, Chemistry, Bioscience and Bioengineering.
- **Research & Facilities Pages:** Office of Research and Development, Central Research Facility, AIoT Fab Facility.
- **Institute News/Announcements (5 items):**Recent announcements and news releases of the institute news portal.

1.1.2 Preprocessing Pipeline

All the documents have been preprocessed in nine steps:

1. **Boilerplate Removal:** HTML `<script>` and `<style>` tags removed via BeautifulSoup before extracting visible text.
2. **Email and URL Removal:** Regular expressions removed email addresses and HTTP/WWW URLs.
3. **Number Removal:** All numeric tokens stripped.
4. **Punctuation Removal:** Non-alphabetic, non-whitespace characters removed.
5. **Lowercasing:** All text converted to lowercase.
6. **Whitespace Normalisation:** Multiple spaces collapsed to single spaces.
7. **Tokenisation:** NLTK `word_tokenize` applied per document.
8. **Stopword & Short-token Removal:** English stopwords removed; tokens shorter than 3 characters dropped.
9. **Domain-specific Filtering:** Tokens `iit`, `jodhpur`, `india` removed to prevent them dominating the embedding space.

1.1.3 Dataset Statistics

Metric	Value
Total Documents	19
Total Tokens (preprocessed)	7,179
Vocabulary Size	1,963 unique tokens

The small vocabulary size reflects the domain-specific, repetitive nature of institutional web content, where terms such as *research*, *faculty*, *engineering*, *publication*, and *student* appear across multiple pages. A word cloud confirmed dominant terms: **research**, **faculty**, **engineering**, **publication**, **student**, **development**, **technology**, and **laboratory**.

1.2 Task 2: Model Training

1.2.1 Model Architectures

CBOW (Continuous Bag of Words). CBOW predicts a target word from its surrounding context. Given context window w, the $2w$ surrounding embeddings are averaged and passed through a linear layer:

$$\hat{\mathbf{v}} = \frac{1}{2w} \sum_{j \neq 0,\, |j| \leq w} \mathbf{e}_{i+j}, \qquad P(w_t \mid \text{context}) = \text{softmax}(\mathbf{W}\hat{\mathbf{v}} + \mathbf{b}) \tag{1}$$

Training minimises cross-entropy loss against the one-hot target.

```python
class CBOW(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.embeddings = nn.Embedding(vocab_size, embed_dim)
        self.linear     = nn.Linear(embed_dim, vocab_size)

    def forward(self, context):
        embeds = self.embeddings(context).mean(dim=1)   # Average context
        return self.linear(embeds)                       # Predict target
```

Listing 1. CBOW Model Architecture

Skip-gram with Negative Sampling. Skip-gram inverts CBOW: given a target word, predict surrounding context words. Negative sampling replaces the full-vocabulary softmax with binary classification. For each positive (target, context) pair, k negatives

are drawn uniformly:

$$\mathcal{L} = -\log \sigma(\mathbf{v}_{w_t}^\top \mathbf{v}'_{w_c}) - \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} \left[\log \sigma(-\mathbf{v}_{w_t}^\top \mathbf{v}'_{w_i}) \right] \tag{2}$$

```python
class SkipGramNS(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.input_embeddings  = nn.Embedding(vocab_size, embed_dim)
        self.output_embeddings = nn.Embedding(vocab_size, embed_dim)

    def forward(self, target, pos_context, neg_context):
        target_emb = self.input_embeddings(target)
        pos_score  = torch.sum(
                            target_emb * self.output_embeddings(pos_context),
                            dim=1)
        neg_score  = torch.bmm(
                            self.output_embeddings(neg_context),
                            target_emb.unsqueeze(2)).squeeze()
        loss = (-torch.log(torch.sigmoid(pos_score))
                - torch.sum(torch.log(torch.sigmoid(-neg_score))))
        return loss.mean()
```

Listing 2. Skip-gram with Negative Sampling

1.2.2 Hyperparameter Experiments

Models trained with Adam (lr $= 0.005$) for 5 epochs, varying: embedding dimension $d \in \{50, 100\}$, context window $w \in \{2, 4\}$, and negative samples $k \in \{5, 10\}$ (Skip-gram only).

CBOW Results.

Table 1. CBOW Training Loss across Hyperparameter Configurations

Dim	Window	Cross-Entropy Loss per Epoch					Final
		Ep.1	Ep.2	Ep.3	Ep.4	Ep.5	
50	2	5.534	3.370	2.281	1.623	1.209	1.209
50	4	5.778	3.642	2.591	1.941	1.502	1.502
100	2	5.348	2.640	1.311	0.754	0.568	0.568
100	4	5.542	2.947	1.642	0.902	0.501	**0.501** ⋆

Skip-gram with Negative Sampling Results.

Table 2. Skip-gram Training Loss across Hyperparameter Configurations

Dim	Win	Neg	Loss per Epoch					Final
			Ep.1	Ep.2	Ep.3	Ep.4	Ep.5	
50	2	5	—	2.561	1.877	1.640	1.561	**1.561** ⋆
50	4	10	8.918	2.993	2.525	2.369	2.297	2.297
100	2	5	14.538	4.905	2.615	1.875	1.580	1.580
100	2	10	20.398	5.107	2.849	2.231	2.065	2.065
100	4	5	10.741	3.678	2.464	2.168	2.101	2.101
100	4	10	13.638	4.127	3.074	2.881	2.871	2.871

1.2.3 Analysis of Hyperparameter Effects

- **Embedding Dimension:** Increasing d from 50 to 100 consistently reduces final loss for both models.Dimensions are more detailed with higher dimensions. representational capacity although find themselves overfitting small corpora.

- **Context Window Size:** A smaller window (where w=2) tends to obtain lower loss. Narrow windows represent smaller syntactic associations; in the case of CBoW,Narrow windows are used to represent smaller syntactic connections; in the case of CBoW, noise is minimised by averaging a smaller number of vectors.

- **Negative Samples:**As k is increased to a range of 5 to$\tilde{1}$0, Skip-gram loss increases. As the vocabulary of negatives is small (1,963 words), the more negatives there are, the greater is the probability of the word being used. noise to signal ratio k=5 is always better.

- **CBOW vs. Skip-gram:** CBOW has significantly better final loss. (0.501 vs. 1.561). The averaging operation gives a more smooth learning. data on small corpora; the negative-sampling model is less robust. with a small vocabulary.

1.3 Task 3: Semantic Analysis

1.3.1 Cosine Similarity — Top-5 Nearest Neighbours

The cosine similarity was applied to retrieve nearest neighbours with the help of embeddings. Top performing Skip-gram model (dim=50, window=2, neg=5):

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|} \tag{3}$$

Table 3. Top-5 Nearest Neighbours via Cosine Similarity (Skip-gram Embeddings)

Query Word	Top-5 Nearest Neighbours
`research`	publication, graduate, passionate, sponsored, invoice
`student`	chapter, wellbeing, placements, alumni, internships
`phd`	teams, alumni, events, ajayagarwal, akag
`exam`	*Not in vocabulary (filtered during preprocessing)*

Discussion.

- **research:** Neighbours pub and graduate are in the scholarly sense of the word. Passionate and sponsored reflect language in faculties.Invoice is a. artefact of administrative pages coexisting with text research funding.
- **student:**Placements, wellbeing of neighbours, alumni, and internships are semantically consistent- the following. show up close to student on institutional Web sites, grabbing the semantic field of student-lifecycle.
- **phd:** Alumni and events are situational 80. reasonable, ajayagarwal and akag are artefacts of named entities. of partial boilerplate removal of faculty profile URLs.
- **exam:** Absent in the ultimate vocabulary. IIT Jodhpur's public pages give attention to research and faculty profiles, but not content of examination - an honest. indicative of a corpus reflection and not an implementation bug.

1.3.2 Analogy Experiments

Computations with the standard vector arithmetic: $\mathbf{v}_{\text{result}} = \mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C \Rightarrow A : B :: C :?$

Table 4. Analogy Experiment Results

Analogy Query	Expected Concept	Top Predicted Words
`research : publication :: analysis : ?`	Methodological output	analysis, sahu, sponsors, multiphysics, programs
`model : data :: algorithm : ?`	Computational input	data, novel, rajpurohit, kinds, based
`learning : model :: training : ?`	Trained artefact	training, cfio, wellbeing, gpio, functionality
`UG : BTech :: PG : ?`	MTech/MSc	*Words not in vocabulary*
`student : exam :: teacher : ?`	—	*exam not in vocabulary*

Discussion. The semantics of the resulting analogy is not classical because of: (1)s̃mall size of corpus -7,179 is by far insufficient compared with millions required to be robust. analogical reasoning; (2)d̃omain narrowness institutional text is not linguistically broad; (3)l̃ack of important academic words btech, mtech, exam, paper from the vocabulary. An entity sahu, rajpurohit, gpio in results certify corpus. noise has passed preprocessing. These are sincere constraints and not enactment.

1.4 Task 4: Visualisation

Two dimensionality-reduction techniques were applied to project embeddings into 2D for 13 vocabulary-confirmed words: *research, publication, analysis, model, data, learning, training, algorithm, method, result, faculty, student, phd*.

Table 5. Visualisation Technique Summary

Technique	Characteristics	Expected Clusters
PCA	Linear; retains maximum variance; `n_components=2`	Research Activity vs. ML/Computational vs. Academic Community
t-SNE	Non-linear; preserves local neighbourhood; `perplexity=5, random_state=42`	Sharper within-cluster groupings; better non-linear separation

- **CBOW (PCA):** Produces compact, tightly grouped clusters due to the averaging operation. Words *model*, *algorithm*, and *training* are expected to cluster closely.
- **Skip-gram (PCA):** More spread-out embeddings as each (target, context) pair

contributes independently—better cluster separation but higher within-cluster variance.

- **t-SNE:** Better reveals non-linear cluster structure. With low perplexity (5), the three expected semantic clusters are more distinguishable than in PCA.
- **Key Observation:** On a small corpus both models produce embeddings reflecting *co-occurrence patterns in institutional text* rather than general semantic meaning.

2 Problem 2: Character-Level Name Generation Using RNN Variants

Objective: Design and compare three sequence models for character-level name generation—Vanilla RNN, Bidirectional LSTM (BLSTM), and RNN with Basic Attention—trained on a dataset of 1,000 Indian names.

2.1 Task 0: Dataset

2.1.1 Dataset Generation

A dataset of 1,000 unique Indian names was generated using an LLM (Claude Sonnet) and stored in `TrainingNames.txt`. Generation covered diverse regional and cultural categories: North India (UP, Bihar, MP, Rajasthan, Haryana, Delhi), South India (Tamil, Telugu, Kannada, Malayalam), West India (Marathi, Gujarati), East India (Bengali, Odia, Assamese), Punjabi/Sikh, Muslim Indian, and South Indian Brahmin names, with a gender balance of approximately 50% male and 50% female.

2.1.2 Data Cleaning

```python
with open("TrainingNames.txt", "r", encoding="utf-8") as f:
    raw_names = [line.strip() for line in f if line.strip()]

names = []
for name in raw_names:
    name = name.lower()
    if name.isalpha() and len(name) >= 3:    # Alphabetic; length >= 3
        names.append(name)

names = list(set(names))    # Remove duplicates
```

Listing 3. Data Loading and Cleaning

Metric	Value
Total Names (after cleaning)	1,000
Minimum Name Length	3 characters
Format	Lowercase alphabetic only
Special Tokens	<pad> (0), <sos> (1), <eos> (2)
Character Vocabulary Size	**29** (26 letters + 3 special)

Sample names: *aarav, anjali, ashok, asmita, kalavati, apoorv, chandra, deepali, aabha,*

anvita.

2.2 Task 1: Model Implementation

2.2.1 Common Setup

Each name is encoded as $\langle \text{sos} \rangle$, c_1, c_2, ..., c_n, $\langle \text{eos} \rangle$. During training the input is $\langle \text{sos} \rangle$ through c_n and the target is c_1 through $\langle \text{eos} \rangle$ (teacher forcing). Sequences are right-padded and padding indices are masked by `CrossEntropyLoss`.

Table 6. Shared Hyperparameters across All Models

Hyperparameter	Value
Embedding Dimension (`EMBED`)	64
Hidden Size (`HIDDEN`)	128
Number of Layers (`LAYERS`)	2
Learning Rate	0.003
Epochs	20
Batch Size	64
Optimiser	Adam
Loss Function	CrossEntropyLoss (`ignore_index=PAD`)
Gradient Clipping	5.0

2.2.2 Model 1: Vanilla RNN

Architecture. An embedding layer feeds a two-layer RNN with dropout (0.2) followed by an FC output layer:

$$\mathbf{h}_t = \tanh\big(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{e}_t + \mathbf{b}_h\big), \qquad \hat{y}_t = \text{softmax}(\mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y)$$

At inference, temperature sampling ($T = 0.8$) and top-k ($k = 5$) are applied.

```python
class VanillaRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, EMBED,
    padding_idx=PAD)
        self.rnn   = nn.RNN(EMBED, HIDDEN, LAYERS,
                            batch_first=True, dropout=0.2)
        self.fc    = nn.Linear(HIDDEN, vocab_size)

    def forward(self, x, h=None):
        x         = self.embed(x)          # (B, T, EMBED)
        out, h    = self.rnn(x, h)         # (B, T, HIDDEN)
        return self.fc(out), h             # (B, T, vocab_size)
```

Listing 4. Vanilla RNN Architecture

Parameter Count.

$$\begin{aligned} \text{Embedding layer} &: 29 \times 64 = 1{,}856 \\ \text{RNN layer 1} &: (64 + 128) \times 128 + 128 = 24{,}704 \\ \text{RNN layer 2} &: (128 + 128) \times 128 + 128 = 32{,}896 \\ \text{FC layer} &: 128 \times 29 + 29 = 3{,}741 \\ \hline \textbf{Total} &: \mathbf{63{,}453} \end{aligned}$$

2.2.3 Model 2: Bidirectional LSTM (BLSTM)

Architecture. Processes sequences in both forward and backward directions, concatenating hidden states:

$$\mathbf{h}_t = \left[\overrightarrow{\mathbf{h}}_t ; \overleftarrow{\mathbf{h}}_t \right] \in \mathbb{R}^{2 \times \text{HIDDEN}}$$

LSTM gating controls information flow via input (**i**), forget (**f**), output (**o**), and cell (**c**) gates. Dropout = 0.3. The FC layer maps from HIDDEN × 2 to `vocab_size`.

```python
class BLSTM(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, EMBED, padding_idx=PAD)
        self.lstm  = nn.LSTM(EMBED, HIDDEN, LAYERS,
                             batch_first=True,
                             bidirectional=True,        # Key difference
                             dropout=0.3)
        self.fc    = nn.Linear(HIDDEN * 2, vocab_size)   # *2 for bidir
```

Listing 5. Bidirectional LSTM Architecture

Parameter Count.

$$\begin{aligned} \text{Embedding} &: 1{,}856 \\ \text{BLSTM layer 1 (bidir)} &: 2 \times 4 \times \big((64 + 128) \times 128 + 128\big) = 197{,}632 \\ \text{BLSTM layer 2 (bidir)} &: 2 \times 4 \times \big((256 + 128) \times 128 + 128\big) = 393{,}216 \\ \text{FC layer} &: 256 \times 29 + 29 = 7{,}453 \\ \hline \textbf{Total} &: \mathbf{603{,}229} \end{aligned}$$

2.2.4 Model 3: RNN with Basic Attention

Architecture. Augments the Vanilla RNN with additive (Bahdanau-style) attention. At each step:

$$e_j = \mathbf{v}^\top \tanh(\mathbf{W}\mathbf{h}_j), \quad \alpha_j = \text{softmax}(e_j), \quad \mathbf{c} = \sum_j \alpha_j \mathbf{h}_j \tag{4}$$

The context vector $\mathbf{c}$ is concatenated with each hidden state before the output layer: $\hat{y}_t = \mathrm{FC}([\mathbf{h}_t; \mathbf{c}])$.

```python
class Attention(nn.Module):
    def __init__(self):
        super().__init__()
        self.attn = nn.Linear(HIDDEN, HIDDEN)
        self.v    = nn.Linear(HIDDEN, 1)

    def forward(self, rnn_out):
        scores  = self.v(torch.tanh(self.attn(rnn_out)))  # (B,T,1)
        weights = torch.softmax(scores, dim=1)            # (B,T,1)
        return torch.sum(weights * rnn_out, dim=1)        # (B,HIDDEN)

class AttnRNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, EMBED, padding_idx=PAD)
        self.rnn   = nn.RNN(EMBED, HIDDEN, LAYERS, batch_first=True)
        self.attn  = Attention()
        self.fc    = nn.Linear(HIDDEN * 2, vocab_size)

    def forward(self, x, h=None):
        x           = self.embed(x)
        rnn_out, h  = self.rnn(x, h)                      # (B,T,HIDDEN)
        context     = self.attn(rnn_out)                  # (B,HIDDEN)
        context     = context.unsqueeze(1).expand_as(rnn_out)
        return self.fc(torch.cat([rnn_out, context], dim=2)), h
```

Listing 6. Attention Module and Attention RNN

Parameter Count.

$$
\begin{aligned}
\text{Embedding} &: 1{,}856 \\
\text{RNN (2 layers)} &: 57{,}600 \\
\text{Attention} &: 128 \times 128 + 128 + 128 \times 1 + 1 = 16{,}641 \\
\text{FC layer} &: 256 \times 29 + 29 = 7{,}453 \\
\hline
\textbf{Total} &: \mathbf{83{,}806}
\end{aligned}
$$

2.2.5 Architecture Summary

Table 7. Model Architecture Comparison

Model	Params	Hidden	Layers	Key Feature
Vanilla RNN	63,453	128	2	Simple recurrence, dropout=0.2
BLSTM	603,229	128	2	Bidirectional LSTM gates, dropout=0.3
Attention RNN	83,806	128	2	RNN + additive attention over hidden states

2.2.6 Training Results

Table 8. Training Loss per Epoch (selected epochs)

Epoch	Vanilla RNN	BLSTM	Attention RNN
1	41.409	39.745	41.052
2	34.114	17.151	31.620
3	32.170	5.033	26.233
5	30.435	0.474	18.870
8	28.319	0.102	12.330
10	27.367	0.063	9.421
15	24.899	0.029	5.096
20	23.055	0.017	2.724

- **BLSTM:** Dramatic loss reduction (39.7 $\rightarrow$ 0.017) indicates **severe overfitting**—the model memorised all 1,000 training names rather than learning generalisable patterns.
- **Vanilla RNN:** Slow, steady descent (41.4 $\rightarrow$ 23.1). This underfitting is, in fact, useful: the model fills in between patterns instead. than memorising.
- **Attention RNN:** Most balanced curve 41.1to2.7, attained. significant minimization with disastrous overfitting.

2.3 Task 2: Quantitative Evaluation

2.3.1 Metrics

- **Novelty Rate:** $\frac{|\{n \in G : n \notin T\}|}{|G|}$—fraction of generated names not in the training set.
- **Diversity:** $\frac{|\text{unique}(G)|}{|G|}$—fraction of unique A name in all generated names.

– **Realism Score:** Heuristic checking length (3–12 chars), vowel presence, and non-repetition ($\max_c \text{count}(c) \leq 3$):

$$\text{Realism}(n) = \mathbf{1}[3 \leq |n| \leq 12] \cdot \mathbf{1}[\exists \text{ vowel}] \cdot \mathbf{1}[\max_c \text{count}(c) \leq 3]$$

2.3.2 Generation Settings

200 names generated per model. Temperatures: RNN=0.8, BLSTM=1.7(increase to interrupt repetition), Attention=0.9. Top-k sampling (k=5) applied.

2.3.3 Quantitative Results

Table 9. Quantitative Evaluation Metrics (200 generated names per model)

Model	Novelty	Diversity	Realism	Params
Vanilla RNN	0.800	0.965	**0.98** ⋆	63K
BLSTM	**1.000**	0.795	0.02	603K
Attention RNN	**1.000**	**0.975** ⋆	0.24	84K

Table 10. Failure Type Analysis per Model

Model	Too Short (<3)	Char Repetition	Non-alpha	Total Failures
Vanilla RNN	0	4	0	4
BLSTM	65	125	26	186
Attention RNN	3	124	0	127

Analysis.

– **Vanilla RNN (best realism):** Realism of 0.98- almost all generated. names are phonologically plausible. 0.80 of novelty is beneficial. memorisation at partiality, without falling.

– **BLSTM (worst realism):** Realism of just 0.02. The model generates a name of some sort–a name of repetitive, degenerate strings. Classic overfitting failure: has learnt training data too well that it cannot generalise.

– **Attention RNN (best diversity):**Best diversity (0.975) and perfect but a failure in character repetitions (124/200) stamps out realism to 0.24.

2.4 Task 3: Qualitative Analysis

2.4.1 Representative Generated Samples

Table 11. Representative Generated Samples per Model

Vanilla RNN	BLSTM	Attention RNN
hemanti	`vivivivivivivivi`	devika
anmali	`gagagagagagagag`	adhai
eshita	`kkkkkkkkkkkkkk`	kusuma
kamala	k *(single char)*	kavali
brijmohan	`oooooooooooooo`	abhitesh
lalendra	*(empty output)*	harinatha
arunath	`jijijijijijijij`	*fyyubuvvvvvvvv*
harimal	`kakakakakakakak`	*gwwuburunananen*

2.4.2 Realism Analysis

Vanilla RNN. Produces alarmingly realistic Indian names. Hemanti, kamala, eshita, brijmohan, harimal, lalendra all are regular in phonology; prefixes hem-, ar-, lal-. Near-misses such as anmali anjali and arunath arun+nath. be creative in interpolating. Only 4 failures out of 200 names.

BLSTM. Generates almost exclusively degenerate outputs: alternating bigram patterns (*vivivi*v, *kakak*), character saturation (*kkkkkkk*, *ooooooo*), and premature termination (*k*, empty strings). Near-zero training loss (0.017) means the model memorised exact character-transition probabilities; at inference any deviation triggers a feedback loop. Even at $T = 1.7$ outputs become chaotic rather than realistic.

Attention RNN. Shows mixed quality. Realistic outputs include *devika, kusuma, abhitesh, harinatha*. However, prefix fixation (many names beginning with *f, g, c*) and character repetition loops (*fyyyyy*..., *ggggg*...) dominate. The global attention context inadvertently reinforces high-weight characters in subsequent predictions.

2.4.3 Failure Mode Taxonomy

Table 12. Failure Mode Taxonomy across Models

Failure Mode	Description	Example	Model(s)
Character loop	Repeatedly predicts same bigram	*viviviv...*	BLSTM, Attn
Saturation	Single char fills entire output	*kkkkkkk*	BLSTM
Premature <eos>	Terminates immediately	k, (empty)	BLSTM
Over-long repeat	Valid prefix + char repetition	*paramohandan*	RNN, Attn
Prefix fixation	All outputs start same char	*f..., g...*	Attention
Fragment blend	Implausible name concatenation	*chizhanchitipie*	Attention

2.4.4 Why Vanilla RNN Outperforms

1. **Capacity matching:** 63K parameters matches the data size (1,000 names). The 603K-parameter BLSTM is over-parameterised and inevitably overfits.

2. **Gradient dynamics:** Indian names average ∼7 characters. Vanishing gradients are less severe for short sequences, so the simple RNN still learns meaningful patterns. LSTM gating provides no advantage here.

3. **Causal inductive bias:** Vanilla RNN is inherently left-to-right, matching autoregressive generation. BLSTM's bidirectionality creates a training-inference mismatch.

4. **Attention side effects:** Attention was designed for long-range dependencies. Short Indian names have few such dependencies, and the global context can inadvertently reinforce the most recent character.

5. **Dataset size:** 1,000 names regularises a 63K-parameter RNN but is insufficient to regularise a 603K-parameter BLSTM.

3 Conclusions

3.1 Problem 1 Summary

Word2Vec embeddings were successfully trained on a 7,179-token corpus from 19 IIT Jodhpur web pages. Key findings:

- CBOW with $d = 100$ and $w = 4$ achieves the lowest training loss (0.501), outperforming all Skip-gram configurations on this small corpus.
- Semantic neighbours for *research*, *student*, and *phd* are partially meaningful but heavily influenced by the narrow institutional nature of the corpus.
- Analogy completion fails on the originally suggested UG/BTech/PG analogies due to vocabulary gaps; limited success is seen for research-domain analogies.
- The primary limitation is corpus size. Future work should include academic regulation documents and course syllabi to enrich vocabulary.

3.2 Problem 2 Summary

Three character-level name generation models were implemented and evaluated on 1,000 Indian names:

Table 13. Final Model Comparison and Ranking

Model	Novelty	Diversity	Realism	Params	Verdict
Vanilla RNN	0.80	0.965	**0.98**	63K	Best overall ⋆
Attention RNN	1.00	**0.975**	0.24	84K	Good diversity; repetition issues
BLSTM	1.00	0.795	0.02	603K	Catastrophic overfit

The key lesson is that **inductive bias alignment matters**: the Vanilla RNN's causal, fixed-capacity design is naturally suited to short-sequence generation from a small dataset. Model complexity must be matched to dataset size—the BLSTM's 603K parameters memorise rather than generalise on 1,000 training examples. The attention mechanism, while theoretically powerful, introduces repetition artefacts when applied to short sequences where long-range dependencies are minimal.